KVM-Based eBPF Rootkit Detection Engine

Milestone two

Luis Abraham - labrahamesco2024@my.fit.edu
Dylin Irons - dirons2024@my.fit.edu
Dominick Morales - dmorales2024@my.fit.edu
Braiden Ames - bames2024@my.fit.edu

Dr. Ribeiro - eribeiro@fit.edu

Client: Amy Alvarez, NASA Scientist

Milestone Completion Matrix:

Task	Completed	Luis	Braiden	Dominick	Dylin
pKVM module VMI	100%	100%	0%	0%	0%
Linux kernel introspection	100%	10%	90%	0%	0%
Initial eBPF introspection	100%	10%	0%	90%	0%
Initial Python bindings	100%	50%	50%	0%	0%
Bindings integration w/ web UI	100%	0%	0%	0%	100%

Discussion of each accomplished task:

1. Developed similar tooling in pKVM to what we used with KVMI to set up reading memory and registers for both the host OS and the hypervisor
2. Map **kallsyms** to virtual memory during a paused state, get process listings (task_struct), analyzing other important objects, and more to aid future eBPF analysis

3. Analyze basic eBPF info like: loaded eBPF programs and basic metadata, eBPF maps and their metadata, program types, IDs, names, and attachment information
4. Create python bindings to milestone 1 work like: VM handling, phys/virt mem/reg r/w, exit hooks, etc. We have CLI tooling for it, so you'd just need to use similar logic.
5. Integrate those python bindings into the web server: VM management and advances introspection settings (this is where you can add the KVMI stuff)

Discussion of each contribution:

- Luis - researched into the AVF and pKVM internals, developed kernel and hypervisor modules that interact with each other, figured out how to naturally circumvent pKVM design decisions that became roadblocks to getting VM introspection working. Got physical memory and register read for the hypervisor, host OS, and guest pVMs all working.
- Braiden - Built the kernel introspection engine that reconstructs a running guest's kernel state from guest memory and vCPU registers alone: derive the kernel page-table root from CR3 (handling KPTI and non-KPTI guests), locate and decode kallsyms for bidirectional symbol address resolution (auto-detecting the table layout and address encoding across kernel versions), and parse the kernel's embedded BTF to read arbitrary struct fields by name instead of hardcoded offsets; all established in one init pass and verified against /proc/kallsyms and pahole.
- Dylin - Python bindings created to create the functionality needed for the dashboard to work on the guest. Dashboard works by getting its data from the bindings which load the C introspection library to get virtual machines as objects. The web server uses those objects to list VMs, read eBPF activity and run detection scans on a set interval. It will send this data to the dashboard to show the user live data.
- Dominick - Did some research into eBPF internals. Created the python modules for eBPF introspection. This allows the user to read, analyze eBPF programs from VM memory while not having anything running inside of the VM. One can decode eBPF programs, disassemble them, and learn about what they are doing. This leads into the next part where the dashboard will be able to take this information and make decisions based on it. Also got a pKVMI testbed running on the side on an Apple Silicon Macbook. The pixel is already an ARM64 testbed, but testing on other ARM64 devices still allows us to expand coverage.

Next Milestone Matrix:

Task	Luis	Dominick	Braiden	Dylin
pKVMI integration to rootview	100%	0%	0%	0%
Isolated pVM for introspection instead kernel module	100%	0%	0%	0%
Porting KVMI to the latest Linux Kernel	0%	100%	0%	0%
Add more eBPF analysis features that the WebUI needs	0%	100%	0%	0%
Research into eBPF rootkits	0%	0%	100%	0%
Start design of specific test cases for eBPF manipulation	0%	0%	100%	0%
Dashboard hidden object detection, flag mismatches and improve information displayed	0%	0%	0%	100%

Discussion of each task:

1. We have the ability to have introspection by poking holes in the AVF on an android phone, now we need to implement rootview handling that support
2. Right now the introspection kernel module that speaks to the hypervisor lives in the guest OS, it's not very secure and doesn’t allow for pausing the host OS. If

we put it in an isolated pVM and have interaction with the vendor module from there, we can allow more functionality.

3. To port KVMI to the latest Linux kernel would allow us to test against modern setups and improve our security stance. The whole reason behind the VM in a VM setup is because we rather would not have been installing old kernels on our host machines. Some programs are broken inside of the VMI VM because of how old the kernel is. In the process of doing this, we can also try to add features that we might be missing (CPU specific).
4. While the eBPF introspection functionality is working now, there are still things that need to be added for a complete picture. The web interface also relies on some features that have not yet been implemented. Once this is done, more advanced information will be available.
5. Researching how rootkits use eBPF will allow us to emulate what a real attacker would do and throw it at our tool.
6. Starting small, we can make small specific eBPF manipulations with binaries to throw at our current tool, putting these binaries together in M4.
7. Comparing the hypervisor view of eBPF programs with the guest own view and flagging mismatches. The dash only lists what exists in the moment, so a person has to decide what is actually suspicious. Add more checks and maybe a history that saves findings across restarts

Client feedback on milestone:

“Looks good to me”

Date(s) of meeting(s) with Faculty Advisor during the current milestone:

22 Oct, 2026

Faculty Advisor feedback on each task for the current Milestone:

1. Good
2. Good
3. Good
4. Good
5. Good
6. Good
7. Good
8. Extra notes - spend some time researching how malware can detect if it is being watched to act differently. Then develop a test malware and test introspection

tool that lives inside the guest to show that our tool being “outside looking in” is very important and is our selling point.

Approval from Faculty Advisor

- **"I have discussed with the team and approved this milestone."**
- **Signature: _____ Date:** 10/22/2026

Luis Abraham	**0**	**1**	**2**	**3**	**4**	**5**	**5.5**	**6**	**6.5**	**7**	**7.5**	**8**	**8.5**	**9**	**9.5**	**10**	10
Braiden Ames	**0**	**1**	**2**	**3**	**4**	**5**	**5.5**	**6**	**6.5**	**7**	**7.5**	**8**	**8.5**	**9**	**9.5**	**10**	10
Dominick Morales	**0**	**1**	**2**	**3**	**4**	**5**	**5.5**	**6**	**6.5**	**7**	**7.5**	**8**	**8.5**	**9**	**9.5**	**10**	10
Dylin Irons	**0**	**1**	**2**	**3**	**4**	**5**	**5.5**	**6**	**6.5**	**7**	**7.5**	**8**	**8.5**	**9**	**9.5**	**10**	10

Facutly Advisor Signature: _____ **Date:** ~~10/22/2026~~